

TOPIC 1- P. Sai Charan Tej-192465048

Experiment 1: Linear Search (Iterative)

main.py	Output
<pre>1 arr = [10, 25, 30, 45, 50] 2 key = 30 3 found = False 4 for i in range(len(arr)): 5 if arr[i] == key: 6 print("Key found at index", i) 7 found = True 8 break 9 if not found: 10 print("Key not found")</pre>	<pre>Key found at index 2 === Code Execution Successful ===</pre>

Experiment 2: Binary Search (Recursive)

main.py	Output
<pre>1 def binary_search(arr, low, high, key): 2 if low <= high: 3 mid = (low + high) // 2 4 if arr[mid] == key: 5 return mid 6 elif key < arr[mid]: 7 return binary_search(arr, low, mid - 1, key) 8 else: 9 return binary_search(arr, mid + 1, high, key) 10 return -1 11 arr = [5, 10, 15, 20, 25] 12 key = 20 13 14 result = binary_search(arr, 0, len(arr) - 1, key) 15 16 if result != -1: 17 print("Key found at index", result) 18 else: 19 print("Key not found")</pre>	<pre>Key found at index 3 === Code Execution Successful ===</pre>

Experiment 3: Factorial

main.py	Output
<pre>1 def iterative_factorial(n): 2 fact = 1 3 for i in range(1, n + 1): 4 fact *= i 5 return fact 6 7 def recursive_factorial(n): 8 if n == 0 or n == 1: 9 return 1 10 return n * recursive_factorial(n - 1) 11 12 n = 5 13 14 print("Iterative Factorial =", iterative_factorial(n)) 15 print("Recursive Factorial =", recursive_factorial(n))</pre>	<pre>Iterative Factorial = 120 Recursive Factorial = 120 === Code Execution Successful ===</pre>

Experiment 4: Fibonacci Series

main.py	Output
<pre>1 n = 6 2 3 a = 0 4 b = 1 5 6 print("Fibonacci Series:") 7 8 for i in range(n): 9 print(a, end=" ") 10 c = a + b 11 a = b 12 b = c</pre>	<pre>Fibonacci Series: 0 1 1 2 3 5 === Code Execution S...</pre>

Experiment 5: Matrix Multiplication

Programiz main.py Python Online Compiler	Output
<pre>1 A = [[1, 2], 2 [3, 4]] 3 B = [[5, 6], 4 [7, 8]] 5 result = [[0, 0], 6 [0, 0]] 7 for i in range(len(A)): 8 for j in range(len(B[0])): 9 for k in range(len(B)): 10 result[i][j] += A[i][k] * B[k][j] 11 print("Result Matrix:") 12 for row in result: 13 print(row)</pre>	<pre>Result Matrix: [19, 22] [43, 50] === Code Execution S...</pre>

Experiment 6: Merge Sort

Programiz main.py Python Online Compiler	Output
<pre>1 def merge_sort(arr): 2 if len(arr) > 1: 3 mid = len(arr) // 2 4 left = arr[:mid] 5 right = arr[mid:] 6 7 merge_sort(left) 8 merge_sort(right) 9 10 i = j = k = 0 11 12 while i < len(left) and j < len(right): 13 if left[i] < right[j]: 14 arr[k] = left[i] 15 i += 1 16 else: 17 arr[k] = right[j] 18 j += 1 19 k += 1 20 21 while i < len(left): 22 arr[k] = left[i]</pre>	<pre>Sorted Array: [3, 9, 10, 27, 38, 43, 82] === Code Execution Successful ===</pre>

Experiment 7: Quick Sort

main.py	Run	Output
<pre>1- def quick_sort(arr): 2- if len(arr) <= 1: 3- return arr 4- 5- pivot = arr[len(arr)//2] 6- 7- left = [x for x in arr if x < pivot] 8- middle = [x for x in arr if x == pivot] 9- right = [x for x in arr if x > pivot] 10- 11- return quick_sort(left) + middle + quick_sort(right) 12- 13- arr = [10, 7, 8, 9, 1, 5] 14- 15- print("Sorted Array:", quick_sort(arr))</pre>		<pre>Sorted Array: [1, 5, 7, 8, 9, 10] === Code Execution Successful ===</pre>

Experiment 8: Tower of Hanoi

main.py	Run	Output
<pre>1- def tower(n, source, auxiliary, destination): 2- if n == 1: 3- print("Move Disk 1 from", source, "to", destination) 4- return 5- 6- tower(n-1, source, destination, auxiliary) 7- 8- print("Move Disk", n, "from", source, "to", destination) 9- 10- tower(n-1, auxiliary, source, destination) 11- 12- n = 3 13- 14- tower(n, 'A', 'B', 'C')</pre>		<pre>Move Disk 1 from A to C Move Disk 2 from A to B Move Disk 1 from C to B Move Disk 3 from A to C Move Disk 1 from B to A Move Disk 2 from B to C Move Disk 1 from A to C === Code Execution Successful ===</pre>

Experiment 9: GCD (Euclidean Algorithm)

main.py	Run	Output
<pre>1 2 3- def gcd(a, b): 4- if b == 0: 5- return a 6- return gcd(b, a % b) 7 8 a = int(input("Enter first number: ")) 9 b = int(input("Enter second number: ")) 10 print("GCD =", gcd(a, b))</pre>		<pre>Enter first number: 3 Enter second number: 4 GCD = 1 === Code Execution Successful ===</pre>

Experiment 10: Insertion Sort

main.py	Run	Output
<pre>1 # Experiment 10: Insertion Sort 2 3- def insertion_sort(arr): 4- for i in range(1, len(arr)): 5- key = arr[i] 6- j = i - 1 7 8- while j >= 0 and arr[j] > key: 9- arr[j + 1] = arr[j] 10- j -= 1 11 12- arr[j + 1] = key 13 14- return arr 15 16 arr = list(map(int, input("Enter elements separated by space: ").split())) 17 18 print("Sorted Array:", insertion_sort(arr))</pre>		<pre>Enter elements separated by space: 1 2 3 6 5 7 Sorted Array: [1, 2, 3, 5, 6, 7] === Code Execution Successful ===</pre>

Experiment 11: Heap Sort

main.py	Run	Output
<pre>1 # Experiment 14: Fast Exponentiation 2 3 def power(x, n): 4 if n == 0: 5 return 1 6 7 half = power(x, n // 2) 8 9 if n % 2 == 0: 10 return half * half 11 else: 12 return x * half * half 13 14 x = int(input("Enter base: ")) 15 n = int(input("Enter exponent: ")) 16 17 print("Result:", power(x, n))</pre>		<pre>Enter base: 2 Enter exponent: 3 Result: 8 === Code Execution Successful ===</pre>

Experiment 15: Closest Pair of Points (Brute Force)

main.py	Run	Output
<pre>1 # Experiment 15: Closest Pair of Points 2 3 import math 4 5 points = [] 6 7 n = int(input("Enter number of points: ")) 8 9 for i in range(n): 10 x, y = map(int, input("Enter x y: ").split()) 11 points.append((x, y)) 12 13 min_dist = float('inf') 14 pair = () 15 16 for i in range(n): 17 for j in range(i + 1, n):</pre>		<pre>Enter number of points: 2 Enter x y: 1 2 Enter x y: 3 4 Closest Pair: ((1, 2), (3, 4)) Distance: 2.8284271247461903 === Code Execution Successful ===</pre>